arrays

1. Linear Search

Find an element by checking each array element one by one. Works on both sorted and unsorted arrays.

```
class Main {
    public static int linearSearch(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89, 23, 43};
        int key = 67;

        System.out.println(linearSearch(arr, key));
    }
}
```

Output: `4`

Points to remember

- Works on sorted and unsorted arrays.
- Start from index `0` and iterate until `i < arr.length`.
- Return `i` when the key is found.
- Return `-1` if the element does not exist.
- Best-case time: $O(1)$; worst-case time: $O(n)$.
- Extra space: $O(1)$.

2. Find the Largest Element

Find the maximum value in an array without sorting it.

```
class Main {
    public static int largest(int[] arr) {
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        return max;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89, 23, 43};

        System.out.println("Largest: " + largest(arr));
    }
}
```

Output: `Largest: 89`

Points to remember

- Initialize the maximum with `Integer.MIN_VALUE` or the first array element.
- Update `max` only when `arr[i] > max`.
- Use `i < arr.length` so the last element is checked.
- Works even when all elements are negative.
- Time: $O(n)$; extra space: $O(1)$.

3. Find the Smallest Element

Find the minimum value in an array without sorting it.

```
class Main {
    public static int smallest(int[] arr) {
        int min = Integer.MAX_VALUE;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
```

```
        }
        return min;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89, 23, 43};

        System.out.println("Smallest: " + smallest(arr));
    }
}
```

Output: `Smallest: 1`

Points to remember

- Initialize the minimum with `Integer.MAX_VALUE` or the first array element.
- Update `min` only when `arr[i] < min`.
- Do not confuse `min` and `max` variable names.
- Works even when all elements are positive or all are negative.
- Time: $O(n)$; extra space: $O(1)$.

4. Binary Search

Find an element in a sorted array by eliminating half of the search space in each iteration.

```
class Main {
    public static int binarySearch(int[] arr, int key) {
        int start = 0;
        int end = arr.length - 1;

        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (arr[mid] == key) {
                return mid;
            } else if (arr[mid] < key) {
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }
        return -1;
```

```
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 23, 43, 67, 89};
        int key = 67;

        System.out.println(binarySearch(arr, key));
    }
}
```

Output: `6`

Points to remember

- The array must be sorted in ascending order for this implementation.
- Initialize `start = 0` and `end = arr.length - 1`.
- Use `while (start <= end)` for standard exact-value search.
- Calculate `mid` using `start + (end - start) / 2` to avoid integer overflow.
- If `arr[mid] == key`, return `mid`.
- If `arr[mid] < key`, move right: `start = mid + 1`.
- If `arr[mid] > key`, move left: `end = mid - 1`.
- Always move a boundary to avoid an infinite loop.
- Return `-1` if the key is not found.
- Standard binary search may return any matching index when duplicates exist.
- Time: $O(\log n)$ worst case; extra space: $O(1)$.

5. Reverse an Array

Reverse the elements of an array in place using the two-pointer approach.

```
class Main {

    public static void reverse_array(int[] arr) {

        int start = 0;

        int end = arr.length - 1;


        while (start < end) {
```

```
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89};

        reverse_array(arr);

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```

Output: `89 67 4 3 2 1`

Points to remember

- Initialize two pointers: `start = 0` and `end = arr.length - 1`.
- Use `while (start < end)` to stop when the pointers meet or cross.

- Swap `arr[start]` and `arr[end]` using a temporary variable.
- Increment `start` and decrement `end` after each swap.
- The original array is modified; no additional array is required.
- Works for odd-length, even-length, single-element, and empty arrays.
- Do not use `start <= end`; swapping the middle element of an odd-length array is unnecessary.
- Time complexity: $O(n)$, since approximately $n/2$ swaps are performed.
- Extra space complexity: $O(1)$, because reversal happens in place.

6. Print All Pairs

Print every possible pair of elements in an array, where each pair contains two elements at different indices and each pair is printed only once.

```
class Main {
    public static void print_pairs(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            int curr = arr[i];

            for (int j = i + 1; j < arr.length; j++) {
                System.out.print("(" + curr + "," + arr[j] + ") ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89};
        print_pairs(arr);
    }
}
```

Output:

```
(1,2) (1,3) (1,4) (1,67) (1,89)
(2,3) (2,4) (2,67) (2,89)
(3,4) (3,67) (3,89)
(4,67) (4,89)
(67,89)
```

Points to remember

- Use nested loops to generate pairs.
- The outer loop selects the first element: `i`.
- The inner loop starts at `j = i + 1` to select the second element.
- Using `j = i + 1` avoids pairs such as `(1,1)` and duplicate reverse pairs such as `(2,1)` after `(1,2)`.
- `System.out.println()` after the inner loop moves to the next line for each first element.
- The number of unique pairs for an array of length n is:
  \[\frac{n(n-1)}{2} \]
- Time complexity: $O(n^2)$, because the number of pairs grows quadratically.
- Extra space complexity: $O(1)$, because no additional data structure is used.

7. Print Subarrays

A subarray is a contiguous part of an array. Your code prints every possible non-empty subarray using three nested loops.

```
class Main {
    public static void print_subarrays(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            int start = i;

            for (int j = i; j < arr.length; j++) {
                int end = j;

                for (int k = start; k <= end; k++) {
                    System.out.print(arr[k] + " ");
                }
                System.out.println();
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89};
        print_subarrays(arr);
    }
```

}

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 67
1 2 3 4 67 89

2
2 3
2 3 4
2 3 4 67
2 3 4 67 89

3
3 4
3 4 67
3 4 67 89

4
4 67
4 67 89

67
67 89

89
```

Points to remember

- A subarray must contain contiguous elements from the original array.
- The outer loop (`i`) selects the starting index.
- The middle loop (`j`) selects the ending index.
- The inner loop (`k`) prints elements from `start` to `end`, inclusive.
- Initialize `j = i` so single-element subarrays are included.
- Use `k <= end` to include the ending element.
- `System.out.println()` after the inner loop moves to the next line.
- The total number of non-empty subarrays of an array of length n is:
  \[\frac{n(n+1)}{2} \]
- For an array of 6 elements, the total number of subarrays is $6(7)/2 = 21$.

- Time complexity: $O(n^3)$ in the worst case, because each subarray's elements are printed.
- Extra space complexity: $O(1)$, since no additional array or collection is created.

Interview tip: Keep these three roles distinct: `i` chooses the start, `j` chooses the end, and `k` traverses the subarray. This pattern is the foundation for subarray sum, maximum subarray, and prefix-sum problems.

8. Find the Maximum Subarray Sum(brute force)

Your code calculates the sum of every possible subarray and finds the maximum subarray sum. This is the brute-force approach using three nested loops.

```
class Main {
    public static void print_maxsubarraysum(int[] arr) {
        int currsum = 0;
        int max_sum = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            int start = i;

            for (int j = i; j < arr.length; j++) {
                int end = j;
                currsum = 0;

                for (int k = start; k <= end; k++) {
                    currsum += arr[k];
                }

                System.out.println(currsum);

                if (currsum > max_sum) {
                    max_sum = currsum;
                }
            }
        }

        System.out.println("The maximum subarray sum is: " + max_sum);
    }
```

```
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89};
        print_maxsubarraysum(arr);
    }
}
```

Output:

```
1
3
6
10
77
166
2
5
9
76
165
3
7
74
163
4
71
160
67
156
89
The maximum subarray sum is: 166
```

The maximum sum is $1 + 2 + 3 + 4 + 67 + 89 = 166$.

Points to remember

- A subarray consists of contiguous elements.
- The outer loop (`i`) selects the starting index.
- The middle loop (`j`) selects the ending index.
- The inner loop (`k`) calculates the sum from `start` to `end`.
- Reset `currsum = 0` for every new subarray.

- Initialize `max_sum = Integer.MIN_VALUE` so arrays containing only negative numbers are handled correctly.
- Update the maximum only when `currsum > max_sum`.
- Every non-empty subarray is considered exactly once.
- The total number of subarrays is:
  \[\frac{n(n+1)}{2} \]
- Time complexity: $O(n^3)$, because each subarray's elements are summed individually.
- Extra space complexity: $O(1)$.

Java DSA Notes: Maximum Subarray Sum Using Prefix Sum

9. Prefix Sum Approach

The prefix sum technique improves the brute-force solution by calculating the sum of each subarray in $O(1)$ time after building a prefix array.

```
class Main {
    public static void print_maxsubarraysum(int[] arr) {
        int currsum = 0;
        int max_sum = Integer.MIN_VALUE;

        int[] prefix = new int[arr.length];
        prefix[0] = arr[0];

        // Build prefix sum array
        for (int i = 1; i < arr.length; i++) {
            prefix[i] = prefix[i - 1] + arr[i];
        }

        // Calculate every subarray sum
        for (int i = 0; i < arr.length; i++) {
            int start = i;

            for (int j = i; j < arr.length; j++) {
                int end = j;

                currsum = (start == 0)
                        ? prefix[end]
```

```
                    : prefix[end] - prefix[start - 1];

                System.out.println(currsum);

                if (currsum > max_sum) {
                    max_sum = currsum;
                }
            }
        }

        System.out.println("The maximum subarray sum is: " + max_sum);
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 67, 89};
        print_maxsubarraysum(arr);
    }
}
```

Output: The same 21 subarray sums as the brute-force approach, followed by:

The maximum subarray sum is: 166

Points to remember

- Prefix sum: `prefix[i]` stores the sum of all elements from index `0` through index `i`.
- Build the prefix array once before checking subarrays.
- Formula for building the prefix array:
  \[prefix[i] = prefix[i-1] + arr[i] \]
- If `start == 0`, the subarray sum is `prefix[end]`.
- Otherwise, calculate the subarray sum using:
  \[sum(i,j)=prefix[j]-prefix[i-1] \]
- The ternary operator `condition ? value1 : value2` selects the appropriate formula.
- Initialize `max_sum` to `Integer.MIN_VALUE` to handle arrays containing only negative values.
- Update `max_sum` whenever `currsum > max_sum`.
- For an empty array, handle the case separately before accessing `arr[0]`.

Kadane’s Algorithm

Your solution works for arrays containing at least one non-negative element, but it fails for an all-negative array because `currsum` resets to `0`.

Corrected code — your approach

```
class Main {
    public static void kadanes(int arr[]) {
        if (arr.length == 0) {
            System.out.println("Array is empty");
            return;
        }

        int currsum = 0;
        int max_sum = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            currsum += arr[i];

            // Update max_sum before resetting currsum
            max_sum = Math.max(currsum, max_sum);

            if (currsum < 0) {
                currsum = 0;
            }
        }

        System.out.println("The maximum subarray sum is: " + max_sum);
    }

    public static void main(String[] args) {
        int arr[] = {-2, -3, 4, -1, -2, 1, 5, -3};
        kadanes(arr);
    }
}
```

Output

The maximum subarray sum is: 7

Points to remember

- `currsum += arr[i]` adds the current element to the running sum.
- `max_sum = Math.max(currsum, max_sum)` records the best sum before resetting.

- If `currsum < 0`, reset it to `0` because a negative running sum cannot improve a future subarray.
- Initializing `max_sum` to `Integer.MIN_VALUE` ensures an all-negative array returns its largest element instead of `0`.

Complexity: Time — $O(n)$; extra space — $O(1)$.

KADANES THAT HANDLES ALL THE CASES:

```
class Main {
    public static void kadanes(int arr[]) {
        if (arr.length == 0) {
            System.out.println("Array is empty");
            return;
        }

        int currsum = 0;
        int max_sum = Integer.MIN_VALUE;

        boolean allNegative = true;

        // Check whether all elements are negative
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] >= 0) {
                allNegative = false;
                break;
            }
        }

        // Handle the all-negative case
        if (allNegative) {
            for (int i = 0; i < arr.length; i++) {
                max_sum = Math.max(max_sum, arr[i]);
            }
            System.out.println("The maximum subarray sum is: " + max_sum);
            return;
        }

        // Standard Kadane's algorithm
        for (int i = 0; i < arr.length; i++) {
            currsum += arr[i];

            max_sum = Math.max(max_sum, currsum);
```

```
            if (currsum < 0) {
                currsum = 0;
            }
        }

        System.out.println("The maximum subarray sum is: " + max_sum);
    }

    public static void main(String[] args) {
        int arr[] = {-2, -3, -4, -1, -5};
        kadanes(arr);
    }
}
```

example:On the first iteration:

`currsum += arr[i]` → `currsum = -2`

`max_sum = Math.max(-2147483648, -2)` → `max_sum = -2`

`currsum < 0` is true, so `currsum = 0`

Now max_sum correctly remembers -2, even though currsum becomes 0.

When the loop reaches -1, max_sum updates to -1, which is the correct answer.

2. What happens if you reverse the order?

```
if (currsum < 0) {

    currsum = 0;

}

max_sum = Math.max(max_sum, currsum);
```

On the first iteration:

1. currsum = -2
2. The condition is true, so currsum = 0
3. max_sum = Math.max(Integer.MIN_VALUE, 0) → max_sum = 0

You have lost the original value -2. The same thing happens for every negative element, so the algorithm incorrectly returns 0.

Remember this rule

- First: Save the best sum found so far in max_sum.
- Then: Reset currsum if it is negative, because a negative running sum cannot help increase the sum of a future subarray.

Trapping Rainwater – Java (Prefix & Suffix Maximum Approach)

```
class Main {


    public static int trappedwater(int height[]) {
        int n = height.length;


        // Step 1: Store the maximum height from the left
        int leftmax[] = new int[n];
        leftmax[0] = height[0];


        for (int i = 1; i < n; i++) {
            // Maximum height till index i from the left
            leftmax[i] = Math.max(height[i], leftmax[i - 1]);
        }
```

```
        // Step 2: Store the maximum height from the right
        int rightmax[] = new int[n];
        rightmax[n - 1] = height[n - 1];

        for (int i = n - 2; i >= 0; i--) {
            // Maximum height till index i from the right
            rightmax[i] = Math.max(height[i], rightmax[i + 1]);
        }

        // Step 3: Calculate the total trapped rainwater
        int trappedwater = 0;

        for (int i = 0; i < n; i++) {
            // Water level is limited by the shorter of the two
boundaries
            int waterlevel = Math.min(leftmax[i], rightmax[i]);

            // Water trapped at index i = water level - bar height
            trappedwater += waterlevel - height[i];
        }

        // Step 4: Return the total trapped water
        return trappedwater;
```

```
    }


    public static void main(String[] args) {
        int height[] = {4, 2, 0, 6, 3, 2, 5};


        System.out.println(trappedwater(height)); // Output: 11
    }
}
```

Note: This version assumes the array is non-empty. For a robust solution, add `if (height.length < 3) return 0;` at the beginning of the method.

Points to remember

1. The formula

$$ \boxed{\text{Water}[i]=\min(L_{\max}[i],R_{\max}[i])-height[i]} $$

- `leftmax[i]` = tallest bar from index `0` to `i`.
- `rightmax[i]` = tallest bar from index `i` to `n-1`.
- `waterlevel` = minimum of the two maximum heights.
- Trapped water = water level minus the current bar's height.

2. Remember the loop directions

Array	Initialization	Loop direction

`leftmax[]`	`leftmax[0] = height[0]`	Left to right
`rightmax[]`	`rightmax[n-1] = height[n-1]`	Right to left
Water calculation	`trappedwater = 0`	Left to right

Memory trick: Left maximum moves forward; right maximum moves backward.

3. Why use `Math.max()` and `Math.min()`?

- `Math.max()` finds the tallest boundary encountered so far.
- `Math.min()` determines the maximum water level because water spills over the shorter boundary.

4. Dry run for the example

Input: `height = {4, 2, 0, 6, 3, 2, 5}`

Index	Height	Left max	Right max	Water trapped
0	4	4	6	0
1	2	4	6	2
2	0	4	6	4

3	6	6	6	0
4	3	6	5	2
5	2	6	5	3
6	5	6	5	0
			Total	11

5. Complexity

- Time: $O(n)$ — three linear traversals.
- Auxiliary space: $O(n)$ — two extra arrays.

Quick revision checklist

- Create arrays of size `n`, not `n - 1`.
- Initialize the first element of `leftmax`.
- Initialize the last element of `rightmax`.
- Use `Math.max()` to build both arrays.
- Use `Math.min()` to calculate the water level.
- Subtract the current bar height.
- Add the water at each index to the total.
- Return and print the total.

Interview tip: The key insight is that the water above a bar depends on the tallest boundary on its left and right—not just its immediate neighbors.

Buy and Sell Stock problem. It returns the maximum profit possible by buying once and selling once, with the buy occurring before the sell.

Corrected code with comments

```
class Main {
    public static int BuyAndSellStocks(int prices[]) {
        int buyprice = Integer.MAX_VALUE;
        int maxprofit = 0;

        for (int i = 0; i < prices.length; i++) {

            // If the current price is greater than the buying price
            if (buyprice < prices[i]) {
                int profit = prices[i] - buyprice;

                // Update the maximum profit
                maxprofit = Math.max(maxprofit, profit);
            } else {
                // Update the minimum buying price
                buyprice = prices[i];
            }
        }

        return maxprofit;
    }

    public static void main(String[] args) {
        int prices[] = {7, 1, 5, 3, 6, 4};
        System.out.println(BuyAndSellStocks(prices));
    }
}
```

Output:

5

Dry run

Day	Price	Minimum buy price	Profit	Maximum profit
0	7	7	—	0

1	1	1	—	0
2	5	1	4	4
3	3	1	2	4
4	6	1	5	5
5	4	1	3	5

The best transaction is to buy at 1 and sell at 6, giving a profit of $6 - 1 = 5$.

Why does your `if-else` work?

- `if (buyprice < prices[i])`: The current price is higher than the minimum buying price, so calculate the profit and update `maxprofit`.
- `else`: The current price is less than or equal to the current buying price, so update `buyprice` to the current price.

Complexity

- Time: $O(n)$ — one traversal of the array.
- Space: $O(1)$ — only a few variables are used.

This also handles decreasing prices, such as `{7, 6, 4, 3, 1}`, returning `0` because no profitable transaction is possible.

Contains Duplicate using only an array by comparing each element with the elements that come after it.

Java solution using an array

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
```

```
            for (int j = i + 1; j < nums.length; j++) {

                // Check if any two elements are equal
                if (nums[i] == nums[j]) {
                    return true;
                }
            }
        }

        return false;
    }
}
```

Dry run

Input: `nums = [1, 2, 3, 1]`

- Compare `1` with `2`, `3`, and `1`.
- When `nums[0] == nums[3]`, a duplicate is found.
- Return `true`.

Complexity

- Time: $O(n^2)$ — nested loops.
- Space: $O(1)$ — no extra data structure.

Note: If you want to use an array and achieve better time complexity, you can sort the array first and then check adjacent elements. That approach takes $O(n \log n)$ time, but modifies the array if you sort it in place.

LeetCode 33. Search in Rotated Sorted Array

Problem Statement:

You are given an integer array `nums` sorted in ascending order with distinct values. Before being passed to your function, the array may have been rotated at an unknown pivot index.

Given the rotated array `nums` and an integer `target`, return the index of `target` if it exists in the array; otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ time complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: `4`

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`

Output: `-1`

Example 3:

Input: `nums = [1]`, `target = 0`

Output: `-1`

Constraints:

- $1 \leq nums.length \leq 5000$
- $-10^4 \leq nums[i] \leq 10^4$
- All values of `nums` are unique.
- `nums` is sorted in ascending order and may be rotated.
- $-10^4 \leq target \leq 10^4$

Java Solution

```
class Solution {
    public int search(int[] nums, int target) {
        int start = 0;
        int end = nums.length - 1;

        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (nums[mid] == target) {
                return mid;
            }

            // Left half is sorted
            if (nums[start] <= nums[mid]) {
                if (nums[start] <= target && target < nums[mid]) {
```

```
                end = mid - 1;
            } else {
                start = mid + 1;
            }
        }
        // Right half is sorted
        else {
            if (nums[mid] < target && target <= nums[end]) {
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }
    }

    return -1;
  }
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Here is the 3Sum brute-force approach using arrays only, without `ArrayList`, `HashSet`, or other collection classes.

3SUM Problem:

```
import java.util.Arrays;

class Solution {
  public int[][] threeSum(int[] nums) {
    int n = nums.length;
    int[][] result = new int[n * n][3];
    int count = 0;

    for (int i = 0; i < n; i++) {
      for (int j = i + 1; j < n; j++) {
        for (int k = j + 1; k < n; k++) {

          if (nums[i] + nums[j] + nums[k] == 0) {
            int[] triplet = {
```

```
                    nums[i], nums[j], nums[k]
                };

                // Sort triplet to identify duplicates
                Arrays.sort(triplet);

                boolean duplicate = false;

                // Check whether triplet already exists
                for (int x = 0; x < count; x++) {
                    if (Arrays.equals(result[x], triplet)) {
                        duplicate = true;
                        break;
                    }
                }

                if (!duplicate) {
                    result[count] = triplet;
                    count++;
                }
            }
        }
    }

    // Return only the valid triplets
    return Arrays.copyOf(result, count);
  }
}
```

How it works

1. Use three nested loops to check every combination of three different indices.
2. If their sum is zero, store the three values in a temporary array.
3. Sort the triplet so that combinations such as `[-1, 2, -1]` and `[-1, -1, 2]` have the same order.
4. Compare the triplet against previously stored results using `Arrays.equals()`.
5. Add it only if it is not a duplicate.
6. Return a copy of the result array containing only valid triplets.

Example

Input: `[-1, 0, 1, 2, -1, -4]`

Output:

[[-1, -1, 2], [-1, 0, 1]]

Complexity

- Time: Up to $O(n^3 + m^2)$, where m is the number of unique triplets found. The duplicate-checking overhead can be significant.
- Space: $O(n^2)$ for the preallocated result array.

Note: This array-based method is useful for learning brute force, but LeetCode 15 expects a `List<List<Integer>>` return type. The method signature must match LeetCode's requirement for submission.